

CLASS 1

Building Blocks of a GenAI Chatbot

From raw text to intelligent conversation — how ChatGPT, Claude & Gemini work under the hood

Conversational AI Applications | IIT Hyderabad



What We'll Cover Today

Six foundational building blocks that power ChatGPT, Claude, and Gemini

01



Tokenisation

How ChatGPT breaks "unbelievable" into ["un", "believ", "able"] — the first step in understanding text

02



Embedding

Converting words into vectors so that "king" - "man" + "woman" = "queen" — how meaning becomes math

03



Model Inference

The forward pass: how a trained transformer turns tokens into the next word prediction

04



Dialogue State

How Claude tracks that "it" refers to your earlier question — managing context across turns

05



Response Gen

Prompt engineering, RAG, and function calling — how the system crafts useful replies

06

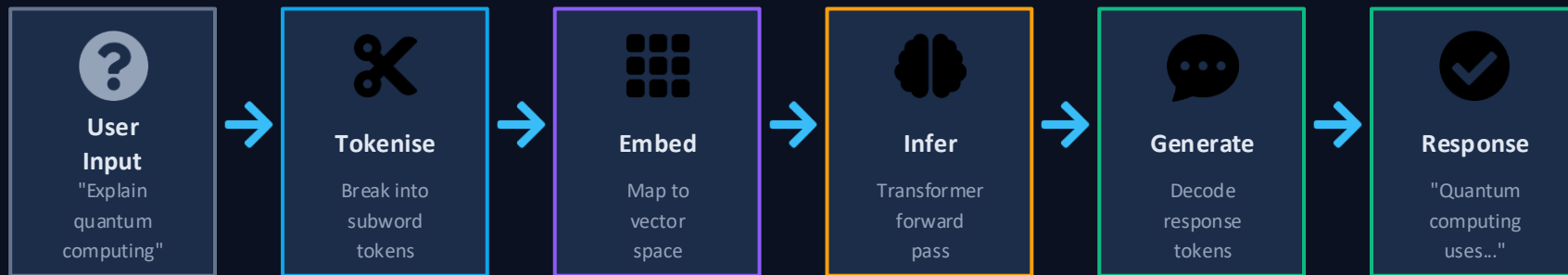


Memory

Why Gemini can remember your preferences across sessions — buffer, summary & vector memory

The Big Picture

What happens when you type a message into ChatGPT?



This pipeline runs in milliseconds. ChatGPT processes ~100K tokens/sec. Claude handles 200K-token context windows. Gemini supports 1M+ tokens. **Today we'll unpack each stage.**

Dialogue State + **Memory** + **Orchestration** wrap around the entire pipeline, managing context across conversation turns

Tokenisation

The first step: breaking language into machine-readable pieces

What is Tokenisation?

Tokenisation splits text into smaller units called tokens — the atomic pieces a language model processes. Unlike simple word splitting, modern tokenisers use statistical algorithms to find optimal subword units.

Tokens can be:

- Whole words: "hello", "world"
- Subwords: "un" + "believ" + "able"
- Characters: "h", "e", "l", "l", "o"
- Bytes: raw byte values (GPT-4)

LIVE EXAMPLE

"ChatGPT is unbelievably fast"



Token IDs:

[16047, 38, 2898, 374, 359, 5765, 2915, 5765]

Vocab Sizes: GPT-4: ~100K tokens | Claude: ~100K | Gemini: 256K tokens | Llama 3: 128K

Tokenisation Methods

Three dominant algorithms — each powering different LLMs

Byte-Pair Encoding (BPE)

GPT-4, Claude, Llama 3

How it works

Iteratively merges the most frequent pair of adjacent tokens. Starts from characters, builds up.

```
"lower" → ["low", "er"]
```

- ✓ Efficient, handles rare words
- ✗ Greedy merging can miss optimal splits

WordPiece

BERT, DistilBERT

How it works

Like BPE but merges the pair that maximises likelihood of the training corpus. Uses ## prefix for continuations.

```
"playing" → ["play", "##ing"]
```

- ✓ Statistically optimal merges
- ✗ Slower training, tied to BERT family

SentencePiece

Gemini, T5, Llama 2

How it works

Treats input as raw byte stream (no pre-tokenisation). Uses either BPE or Unigram model. Language-agnostic.

```
"hello" → ["_hel", "lo"]
```

- ✓ Works for any language/script
- ✗ Larger vocab needed for byte-level

BPE Deep Dive

The algorithm behind GPT-4 and Claude's tokeniser

Algorithm Steps

- 1 Start with character-level vocabulary
{a, b, c, d, ...}
- 2 Count all adjacent pair frequencies
in the training corpus
- 3 Merge the most frequent pair into
a new token
- 4 Repeat until target vocab size
is reached (e.g., 100K for GPT-4)

Worked Example: "aaabdaaabac"

Step	Most Frequent	Merged	Result
Init	—	—	a a a b d a a b a c
1	(a,a) → Z	Z = aa	Z a b d Z a b a c
2	(Z,a) → Y	Y = Za = aaa	Y b d Y b a c
3	(Y,b) → X	X = Yb = aaab	X d X a c

Final: "XdXac" — 5 tokens from 11 characters = **2.2x compression**

WordPiece Deep Dive

BERT's tokeniser — maximising corpus likelihood instead of frequency

Key Difference from BPE

Instead of merging the most frequent pair, WordPiece merges the pair that maximises the likelihood of the training data.

$$\text{score}(a,b) = \text{freq}(ab) / (\text{freq}(a) \times \text{freq}(b))$$

The ## prefix marks continuation tokens:

"unbelievable"

→ ["un", "##believ", "##able"]

The ## tells the model this piece continues a previous token

BPE vs WordPiece

Aspect	BPE	WordPiece
Merge criterion	Most frequent pair	Max likelihood gain
Prefix marker	None	## for continuations
Used by	GPT-4, Claude, Llama	BERT, DistilBERT
Speed	Fast to train	Slower (likelihood calc)
Strength	Simple & effective	Statistically optimal

SentencePiece Deep Dive

Gemini's tokeniser — language-agnostic, byte-level processing

Three Key Features

1

No Pre-tokenisation

Treats input as a raw byte stream. Whitespace is just another character (`_` = space). No language-specific rules needed.

2

Two Sub-algorithms

Can use either BPE or Unigram model internally. Unigram starts with a large vocab and prunes tokens that least reduce corpus likelihood.

3

Truly Language-agnostic

Works identically for English, Hindi, Japanese, Arabic. This is why Gemini handles 100+ languages with a single 256K-token vocabulary.

BPE vs Unigram inside SentencePiece

	BPE mode	Unigram mode
Direction	Bottom-up merging	Top-down pruning
Start	Characters → merge up	Large vocab → prune
Used by	Llama 2	T5, Gemini (256K vocab)

The `_` (underscore) convention

"Hello world" → ["_Hello", "_world"]

"Helloworld" → ["_Hello", "world"]

`_` marks word boundaries — no separate whitespace tokenisation needed

Embeddings

Turning words into vectors that capture meaning — the language of math

What are Embeddings?

Each token maps to a dense vector (list of numbers) in a high-dimensional space. Words with similar meanings end up near each other — "happy" is close to "joyful" but far from "table".

Dimensions: GPT-4: 12,288d | Claude: 8,192d | BERT: 768d

Example vectors (simplified to 3D):

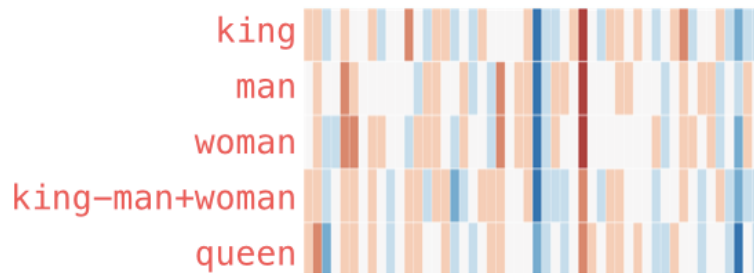
"king" → [0.8, 0.2, 0.9]

"queen" → [0.7, 0.8, 0.9]

"table" → [0.1, 0.3, 0.0]

The Famous Word Analogy

king - man + woman ≈ queen



king - man + woman ≈ queen

Vector arithmetic captures semantic relationships!

Embedding Properties

What makes embeddings powerful for AI?



High Dimensionality

Each word maps to 768-12,288 numbers. More dimensions = more nuance. GPT-4 uses 12,288 dimensions per token.



Semantic Similarity

$\text{cosine}(\text{"happy"}, \text{"joyful"}) \approx 0.92$ but $\text{cosine}(\text{"happy"}, \text{"table"}) \approx 0.15$.
Nearby vectors = related concepts.



Arithmetic Relationships

"Paris" - "France" + "Japan" $\approx$ "Tokyo". Linear algebra captures analogies automatically.



Learned, Not Hardcoded

Embeddings are trained on billions of tokens. The model discovers that "puppy" $\approx$ "kitten" from context alone.

IN PRACTICE Claude uses embeddings to understand that "I need help with my code" and "debug my program" mean the same thing

Model Inference

Inside the transformer: how tokens become predictions

The Inference Pipeline

1 Input Processing

Token IDs + positional encoding → input matrix

2 Self-Attention (Q·K·V)

Each token attends to every other token — capturing context

3 Feed-Forward Layers

Non-linear transformation through MLP networks

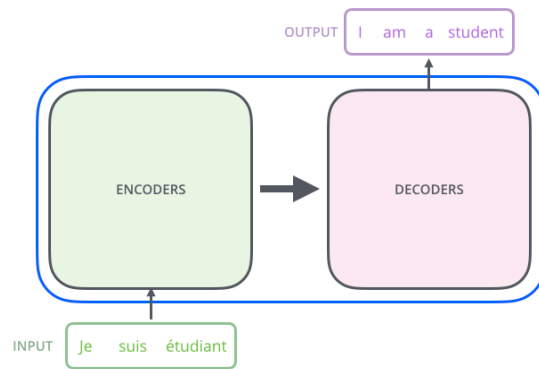
4 Layer Stacking

GPT-4: 120 layers. Claude: ~80 layers. Repeat attention + FFN.

5 Output Logits

Final layer produces probability over entire vocabulary

Transformer Architecture



Source: Jay Alammar — *The Illustrated Transformer*

Scale: GPT-4: ~1.8T params | Claude 3.5: undisclosed | Gemini Ultra: ~1.5T

Decoding Strategies

How the model picks the next word from thousands of candidates

Greedy

—

Always pick the highest-probability token. Fast but repetitive and boring.

Top-k

$k = 40$

Sample from the k most likely tokens. $k=40$ is a common default in GPT models.

Top-p (Nucleus)

$p = 0.95$

Sample from tokens whose cumulative probability exceeds p . Adapts candidate pool dynamically.

Temperature

$T = 0.7$

Scales logits before softmax. $T < 1$ = focused/deterministic. $T > 1$ = creative/random.

IN PRACTICE ChatGPT defaults to $T=1$, $\text{top}_p=1$. Claude uses $T=1$ by default. Most production systems use $T=0.7$ + $\text{top}_p=0.95$ for a balance of coherence and variety.

Dialogue State Management

Keeping track of the conversation so responses stay relevant

WHY DO WE NEED IT?

Without state management, every user message is processed in isolation — the model has no memory of what was said before.

Dialogue state tracks:

- Intent** what the user wants (book, cancel, ask)
- Entities** names, dates, locations, quantities
- History** what was said in prior turns
- Actions** what the bot already did (API calls, confirmations)

EXAMPLE: FLIGHT BOOKING

User: "Book me a flight to Paris"

intent=book_flight, dest=Paris

Bot: "When would you like to travel?"

slot: date=MISSING

User: "Next Friday"

date=next_friday ✓

Bot: "3 flights found. Which do you prefer?"

awaiting: selection

Components of Dialogue State

The four building blocks every conversational system tracks



Intent Recognition

Classifies user's goal each turn: `book_flight`, `cancel_order`, `ask_weather`. Modern LLMs do this zero-shot — no separate NLU model needed.

GPT-4, Claude: implicit in prompt



Entity / Slot Extraction

Extracts parameters: dates, names, locations. Can be required (destination) or optional (seat preference). LLMs extract via function calling.

Rasa NLU, spaCy, GPT function args



Conversation History

The full or windowed turn history. For LLMs, this IS the prompt. Older turns may be summarised to fit context limits.

Claude: 200K tokens, GPT-4: 128K



System Actions Log

What the bot has done: APIs called, confirmations sent, errors hit. Prevents duplicate actions and enables recovery.

Temporal event history, LangSmith

The system asks targeted questions until all required slots are filled

SLOT TABLE

Slot	Required?	Status
destination	Yes	Paris ✓
date	Yes	next_friday ✓
passengers	Yes	MISSING ✗
class	No	economy (default)
return_date	No	MISSING

SLOT FILLING LOOP

1. Parse user message & extract entities

2. Update slot values in state

3. Check: any required slots missing?

YES → Ask for missing slot

NO → Execute action!

Traditional vs LLM-Based DSM

Two paradigms — and why modern systems combine both

Traditional (Rule-Based)

Explicit slot definitions

Hand-craft every entity, intent, and dialogue path

Finite state machines / frames

Rigid flowcharts define conversation transitions

Separate NLU pipeline

Dedicated intent + entity models (Rasa, Dialogflow)

Predictable but brittle

Can't handle unexpected user inputs gracefully

LLM-Based (Modern)

State inferred from context window

The prompt IS the state — no explicit slot defs needed

Zero-shot intent + entity parsing

GPT-4/Claude understand intents without training data

Function calling as actions

LLM decides WHEN and HOW to call external tools

Flexible but needs guardrails

Can hallucinate state — structured output helps

IN PRACTICE Most production chatbots use hybrid: LLMs for understanding + structured state for reliability. Anthropic's Claude uses `tool_use` for structured actions.

In LLM chatbots, the entire conversation prompt IS the state

WHAT THE LLM ACTUALLY SEES

[System] You are a helpful travel assistant.

[User] Book me a flight to Paris.

[Assistant] When would you like to travel?

[User] Next Friday.

[Tool Call] `search_flights(dest='Paris', date='Mar 13')`

[Tool Result] `{flights: [{id: 'AF123', price: '€240'}...]}`

[Assistant] I found 3 flights. AF123 at €240 is cheapest...

KEY CHALLENGES

Token Limits

Context has a fixed size:

GPT-4: 128K, Claude: 200K,

Gemini: 1M+ tokens

Cost Scaling

Longer prompts = more tokens

= higher cost per API call.

\$0.01–0.06 per 1K tokens

Summarisation

Old turns must be compressed.

Risk: losing critical details

like confirmed bookings

Hallucination

LLM may "forget" or fabricate

earlier context. Structured

state stores prevent this

IN PRACTICE Claude manages state with 200K-token windows. ChatGPT's memory feature extracts facts to a persistent store. Gemini can hold 1M+ tokens — entire codebases.

Workflow Orchestration for State

Using systems like Temporal to make dialogue flows durable and observable

THE PROBLEM

Real chatbots call external APIs (flights, payments, databases). Any step can fail, timeout, or need retries.

If the process crashes mid-conversation, naive state management loses everything. Users must start over.

THE SOLUTION

Workflow engines (Temporal, AWS Step Functions, Inngest) persist every step of the dialogue.

If a server crashes, the workflow resumes exactly where it left off — no lost state, no duplicate actions.

HOW IT MAPS TO CHATBOT STATE

Workflow = One conversation session

Each user conversation is a long-running workflow that may span hours or days.

Activities = External API calls

`search_flights()`, `book_hotel()`, `charge_payment()` — each is a retryable activity.

State = Dialogue state (slots)

Destination, dates, preferences — persisted automatically by the engine.

Signals = User messages

Each new user turn is a signal that advances the workflow to its next step.

IN PRACTICE Anthropic uses workflow orchestration internally. Uber's chatbot runs on Cadence (Temporal predecessor). Amazon uses Step Functions for Alexa skills.

A flight-booking conversation as a durable workflow

PSEUDO-CODE (Temporal-style)

```
async function bookFlightWorkflow(userId):  
  # Step 1: Greet & collect destination  
  dest = await waitForSignal('user_msg')  
  state.dest = extractEntity(dest)  
  
  # Step 2: Collect date (retries if invalid)  
  date = await waitForSignal('user_msg')  
  state.date = parseDate(date)  
  
  # Step 3: Search flights (auto-retry)  
  flights = await activity.searchFlights(state)  
  sendMessage(user, formatResults(flights))  
  
  # Step 4: Book selected flight  
  choice = await waitForSignal('user_msg')  
  booking = await activity.bookFlight(choice)  
  return booking # Workflow complete ✓
```

KEY BENEFITS

Crash Recovery

Server dies mid-booking?
Resumes at exact step.

Auto Retries

API timeout on search?
Retried with backoff.

Full Observability

Every state transition logged.
Debug any turn.

Long-Running State

Conversation spans days?
State durably persisted.

Versioning

Deploy new flow without
breaking in-progress chats.

IN PRACTICE Temporal turns your chatbot's dialogue flow into a reliable state machine — for free. Used by Netflix, Uber, Stripe for mission-critical workflows.

Response Generation

How the chatbot turns understanding into a fluent, relevant reply

THE GENERATION PIPELINE

- 1 Receive context + dialogue state
- 2 Construct full prompt: system + history + current query + tools
- 3 LLM forward pass: logits over vocabulary (~100K tokens)
- 4 Apply decoding: greedy, top-k, top-p, temperature
- 5 Post-process: safety filters, format checks, grounding

TWO PARADIGMS

Retrieval-Based

Selects best response from a pre-defined set or knowledge base. Fast & predictable, but limited flexibility. Used in FAQ bots and early Alexa skills.

Generative (LLM)

Creates novel text token-by-token. Flexible & creative, but needs safety filters and grounding. Used in ChatGPT, Claude, Gemini — all modern chatbots.

IN PRACTICE All modern chatbots (ChatGPT, Claude, Gemini) use generative approaches. Hybrid systems use retrieval to ground the generation (RAG).

The system prompt shapes how the model generates replies

EXAMPLE SYSTEM PROMPT

You are TravelBot, a friendly travel assistant.

RULES:

- Always be polite and concise
- If unsure, say "I'm not sure"
- Never make up flight numbers
- Use bullet points for options
- Confirm details before booking

TOOLS AVAILABLE:

- search_flights(dest, date, pax)
- book_flight(flight_id)
- get_weather(city, date)

PROMPT COMPONENTS

Persona

Sets tone, personality,
and domain expertise

Rules / Guardrails

Constrains behaviour:
safety, accuracy, format

Tools / APIs

Defines what actions the
model can invoke

Few-Shot Examples

Demos of ideal responses
set the output quality bar

IN PRACTICE Claude's system prompt can be 200K tokens. GPT-4 supports structured output schemas. Gemini uses 'system instructions' with grounding configs.

RAG: Retrieval-Augmented Generation

Grounding LLM responses in real, up-to-date information

1. Retrieve

User query is embedded and used to search a vector DB for relevant documents.

Tools: Pinecone, Weaviate, ChromaDB, pgvector



2. Augment

Retrieved context is inserted into the prompt alongside the user question.

This gives the LLM fresh, verified information.



3. Generate

LLM generates a response grounded in the retrieved docs.

Can cite sources — reducing hallucination by 80-90%.

Safety & Post-Processing

Guardrails between the model and the user

SAFETY LAYERS



Input Filtering

Block prompt injections, PII detection, content classification before model sees input



System Prompt

Persona constraints, topic boundaries, refusal patterns baked into the prompt



Output Filtering

Toxicity scoring, PII redaction, hallucination detection, citation verification



Human Review

Escalation to agents for edge cases, feedback loops for model improvement

POST-PROCESSING

1

Format & Structure

Markdown → rich text, JSON extraction, code block formatting, list rendering

2

Citation Injection

Attach source URLs, footnotes, confidence scores to factual claims in response

3

Tone Adjustment

Brand voice enforcement, formality level, emoji/casual filters based on context

4

Streaming & Chunking

Token-by-token SSE delivery, sentence boundary buffering, typing indicators

05

Function Calling & Tool Use

LLMs as reasoning engines that invoke external tools



WORKED EXAMPLE

User: "What's the weather in Paris next Friday?"

LLM → `{ "function": "get_weather", "args": { "city": "Paris", "date": "2026-03-13" } }`

Tool → `{ "temp": "14°C", "condition": "partly cloudy", "rain_chance": "20%" }`

LLM: "Paris will be around 14°C next Friday with partly cloudy skies – light layers should be fine!"

IN PRACTICE ChatGPT Plugins & GPT Actions. Claude's tool_use API. Gemini function calling. Open standard: Model Context Protocol (MCP) by Anthropic.

Memory in Conversational AI

Going beyond the context window — remembering across conversations

THE PROBLEM

LLMs are stateless — each API call starts fresh.

The context window holds the current conversation, but once it ends, everything is forgotten.

A user who chatted yesterday is a stranger today.

This is unlike human conversation, where we naturally remember preferences, history, and relationships over time.

WHAT MEMORY ENABLES

Personalisation

"You prefer window seats and vegetarian meals"

Continuity

"Last time we discussed your trip to Tokyo"

Learning

"You've asked about Python errors 5 times — here's a guide"

Efficiency

"I already have your billing address on file"

Memory Mechanisms

Four approaches — from simple to sophisticated

Conversation Buffer

Store the entire conversation history and feed it back into the prompt each turn.

✓ Simple, lossless, easy to debug

✗ Hits token limits fast (~128k tokens max)

Summary Memory

Periodically summarise older turns into a compressed paragraph. Keep recent turns verbatim.

✓ Scalable, retains the gist of past context

✗ Loses detail, summarisation errors accumulate

Entity / Fact Store

Extract structured facts (user prefs, entities) into a key-value or graph database.

✓ Precise, queryable, token-efficient

✗ Extraction can miss nuance and relationships

Vector / Semantic Memory

Embed conversation chunks and store in a vector DB. Retrieve relevant memories via similarity search.

✓ Flexible, handles large histories at scale

✗ Retrieval relevance varies, needs tuning

IN PRACTICE LangChain offers all 4 types via ConversationBufferMemory, ConversationSummaryMemory, EntityMemory, VectorStoreRetrieverMemory.

Hybrid & Tiered Memory

Production chatbots combine multiple memory types into tiers

Working Memory

Current turn



Context window with last N messages + system prompt.
Analogy: your desk with papers currently being worked on.

Short-Term Memory

Current session



Buffer or summary of this conversation, stored in-memory or session DB.
Analogy: your notes from today's meeting.

Long-Term Memory

Across sessions



Vector DB + entity store: user prefs, past facts, conversation embeddings.
Analogy: your filing cabinet with folders for each client.

Example: User says "Book my usual flight" → Working memory has the request → Short-term has today's context → Long-term retrieves "prefers Indigo, window seat, vegetarian"

Memory Read & Write Patterns

When and how memories are stored and retrieved

WRITING MEMORIES

After each turn

Embed the turn and upsert into vector DB.
Extract entities and update fact store.

End of session

Generate a session summary.
Store as a single retrievable document.

Explicit save

User says "Remember that I prefer..."
Directly writes to long-term store.

Background distillation

Periodically re-summarise and consolidate
old memories to save storage and tokens.

READING MEMORIES

Semantic retrieval

Embed current query, find top-k similar
past memories from vector DB.

Recency-weighted

Prioritise recent memories over old ones.
Use time-decay scoring in retrieval.

Entity lookup

Query fact store: "What are this user's
preferences for flights?"

Hybrid retrieval

Combine vector similarity + keyword search
+ entity lookup for best recall.

IN PRACTICE ChatGPT Memory writes explicitly + distills. Claude Projects uses long-context injection. Zep, Mem0, and MemGPT implement hybrid read/write patterns.

Evaluating Memory Systems

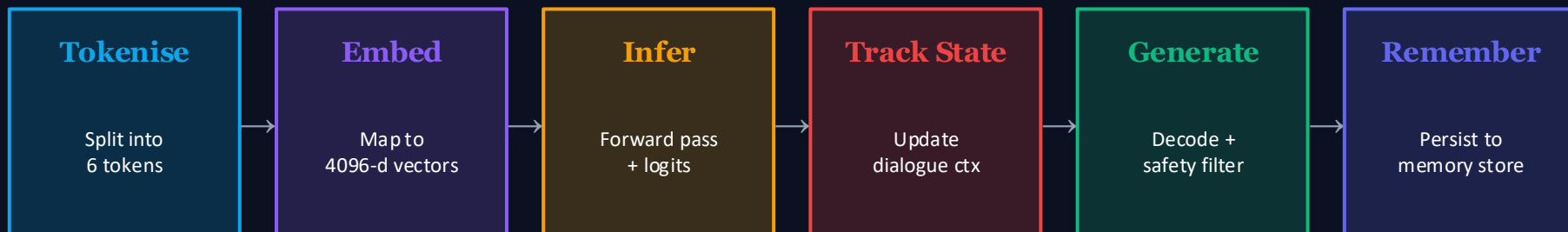
How to measure whether your memory approach actually works

Metric	What it Measures	How to Test
Recall Accuracy	Does the system retrieve the right memory when needed?	Ask about past preferences; check if correct memory surfaces.
Precision	Are retrieved memories relevant, or is there noise?	Count irrelevant memories in top-k results across test queries.
Latency	How much time does memory retrieval add to response time?	Benchmark p50/p95 retrieval time under realistic load.
Token Cost	How many tokens does injected memory consume in the prompt?	Measure avg tokens added per turn. Compare to total context budget.
User Satisfaction	Does memory actually improve the conversation experience?	A/B test: memory ON vs OFF. Measure task completion + NPS.
Staleness	Are outdated memories being used over current information?	Update a fact, then check if the bot uses the new vs old version.

Start simple (buffer), measure these metrics, then add complexity (vector + entity) only where the numbers justify it.

Putting It All Together

User types: "What is machine learning?"



Output: "Machine learning is a subset of AI that enables systems to learn from data..."

Generated one token at a time via autoregressive decoding, each feeding back as input for the next token

The dialogue state and memory persist across turns. Each new user message re-enters this pipeline with accumulated context — this is what makes it a conversation rather than isolated Q&A.

Key Takeaways

- 1 Tokenisation**
Subword methods (BPE, WordPiece, SentencePiece) split text into model-digestible pieces — the foundation of every LLM.
- 2 Embedding**
Dense vectors capture meaning geometrically — similar concepts cluster together. King – Man + Woman ≈ Queen.
- 3 Model Inference**
Transformers process tokens through self-attention layers; decoding strategies (greedy, beam, top-k/p) shape output quality.
- 4 Dialogue State**
Tracks intent, entities, and history so the bot maintains coherent multi-turn conversations — traditionally or via LLM context.
- 5 Response Gen**
Prompt engineering, RAG, tool use, and safety filters turn model understanding into fluent, grounded, safe replies.
- 6 Memory**
Buffer, summary, entity, and vector stores let the chatbot remember across sessions — turning Q&A into relationships.



Next Class: Attention Mechanisms, Transformer Architecture Deep-Dive, and Fine-tuning

Questions?

Let's discuss what we covered today

Tokenisation

Embedding

Inference

Dialogue State

Response Gen

Memory

